ENSIAS

Université Mohammed V de Rabat
École Nationale Supérieure
d'Informatique et d'Analyse des Systèmes

جامعة محمد الخامس بالرباط
Université Mohammed V de Rabat

Deep Learning – Lecture 5

Régularisation : weight decay, Dropout et Batch Normalization

Pr. Aissam BERRAHOU

ENSIAS – Université Mohammed V de Rabat

Module Deep Learning – 2026-2027

Objectifs de la lecture

- reconnaître le surapprentissage sur des courbes d'apprentissage ;
- écrire la perte L2/L1 et la mise à jour *weight decay* ;
- expliquer le Dropout à l'entraînement et à l'inférence ;
- écrire les équations de la Batch Normalization ; connaître la Layer Normalization ;
- utiliser augmentation de données et arrêt précoce ;
- combiner ces techniques dans Keras.

Plan de la lecture

1. Surapprentissage
2. Pénalités sur les poids
3. Dropout
4. Normalisation
5. Augmentation de données et arrêt précoce

Plan

1. Surapprentissage

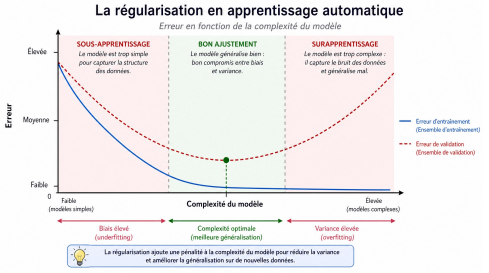
2. Pénalités sur les poids

3. Dropout

4. Normalisation

5. Augmentation de données et arrêt précoce

Biais, variance, surapprentissage



- Erreur $=$ biais^2 $+$ variance $+$ bruit : la régularisation réduit la variance.
- Démarche : (1) obtenir un modèle capable de surapprendre ; (2) régulariser.

Plan

1. Surapprentissage

2. Pénalités sur les poids

3. Dropout

4. Normalisation

5. Augmentation de données et arrêt précoce

Régularisation L2 = weight decay

Perte et mise à jour

$\tilde{J} = J + \frac{\lambda}{2}\sum_j \theta_j^2 \;\Rightarrow\; \theta \leftarrow (1-\eta\lambda)\theta - \eta\nabla J$

- Chaque pas « rétrécit » les poids ; fonction plus lisse.
- Biais non régularisés ; $\lambda \approx 10^{-4}$.

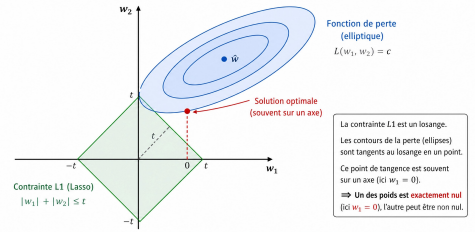
Effet

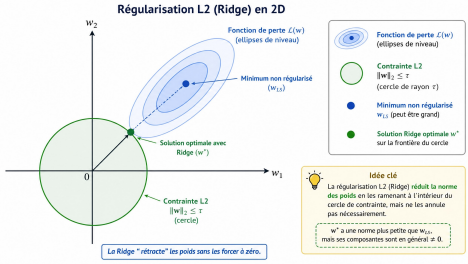
$\eta = 0{,}1$, $\lambda = 0{,}01$: facteur 0,999 par pas ; après 1 000 pas : 37 % (si le gradient ne le soutient pas).

L1

$\lambda \sum |\theta_j|$: gradient constant, pousse des poids à **zéro** (parcimonie).

L1 vs L2


w_2
$\hat{w}$
Fonction de perte (elliptique)
$L(w_1, w_2) = c$
t
Solution optimale (souvent sur un axe)
$-t$
0
t
w_1
$-t$
Contrainte L1 (Lasso)
$|w_1| + |w_2| \leq t$
La contrainte $L1$ est un losange.
Les contours de la perte (ellipses) sont tangents au losange en un point.
Ce point de tangence est souvent sur un axe (ici $w_1 = 0$).
⇒ Un des poids est exactement nul (ici $w_1 = 0$), l'autre peut être non nul.
La régularisation L1 (Lasso) favorise la parcimonie : elle pousse la solution sur les axes, forçant souvent un ou plusieurs poids à être exactement nuls.


Régularisation L2 (Ridge) en 2D
w_2
Fonction de perte $\mathcal{L}(w)$ (ellipses de niveau)
Minimum non régularisé (w_{LS})
Solution optimale avec Ridge (w^*)
0
w_1
Contrainte L2 $\|w\|_2 \leq \tau$ (cercle)
La Ridge " rétracte" les poids sans les forcer à zéro.
Fonction de perte $\mathcal{L}(w)$ (ellipses de niveau)
Contrainte L2 $\|w\|_2 \leq \tau$ (cercle de rayon τ)
Minimum non régularisé w_{LS} (peut être grand)
Solution Ridge optimale w^* sur la frontière du cercle
Idée clé
La régularisation L2 (Ridge) réduit la norme des poids en les ramenant à l'intérieur du cercle de contrainte, mais ne les annule pas nécessairement.
w^* a une norme plus petite que w_{LS}, mais ses composantes sont en général $\neq 0$.

Plan

1. Surapprentissage

2. Pénalités sur les poids

3. Dropout

4. Normalisation

5. Augmentation de données et arrêt précoce

Dropout

Principe (inverted dropout)

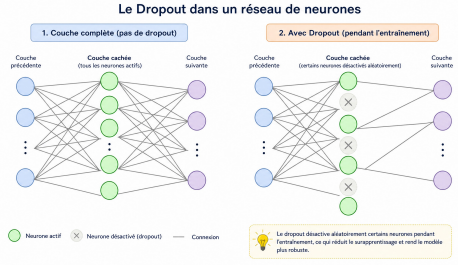
$\mathbf{m} \sim \text{Bernoulli}(1-p), \quad \tilde{\mathbf{a}} = \dfrac{\mathbf{m} \odot \mathbf{a}}{1-p}$

Désactivé à l'inférence.

- empêche la co-adaptation ;
- ensemble implicite de sous-réseaux ;
- $p = 0{,}5$ (denses), 0,1–0,3 (conv, RNN).

Exemple

$\mathbf{a} = (2;1;4;3)$, $p = 0{,}5$, $\mathbf{m} = (1;0;0;1)$
$\Rightarrow (4;0;0;6)$



Plan

1. Surapprentissage

2. Pénalités sur les poids

3. Dropout

4. Normalisation

5. Augmentation de données et arrêt précoce

Batch Normalization

Sur un mini-lot

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \qquad y_i = \gamma \hat{x}_i + \beta \quad (\gamma, \beta \text{ appris})$$

- Inférence : moyennes mobiles de μ et σ^2.
- Stabilise, accélère, autorise des η plus grands, léger effet régularisant.

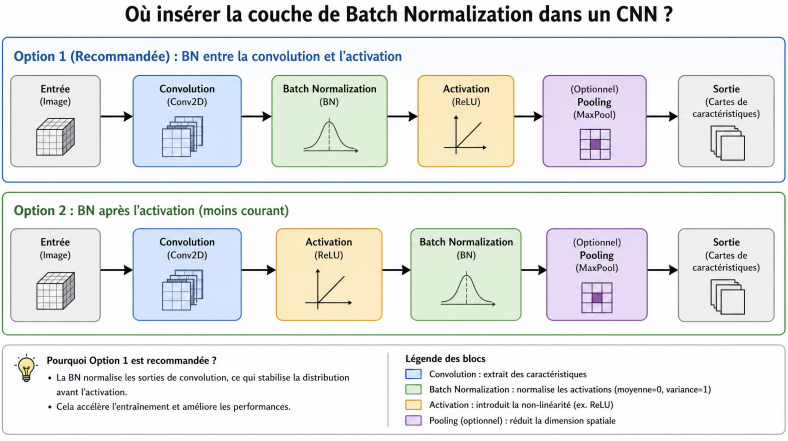
Lot $(2; 4; 6; 8)$, $\gamma = 2$, $\beta = 1$

$\mu = 5$, $\sigma^2 = 5$, $\hat{x} = (-1{,}34; -0{,}45; 0{,}45; 1{,}34)$, $y = (-1{,}68; 0{,}11; 1{,}89; 3{,}68)$

Limites

Petits lots, séquences variables $\Rightarrow$ **Layer Normalization** (sur les caractéristiques de chaque exemple) : standard des Transformers.

Où placer la Batch Normalization ?



Plan

1. Surapprentissage

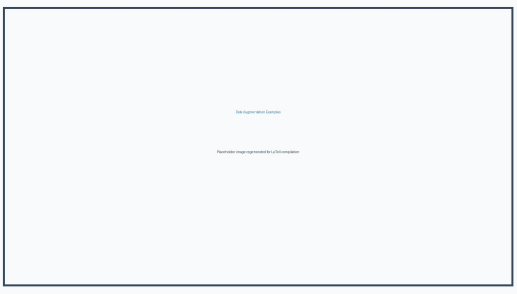
2. Pénalités sur les poids

3. Dropout

4. Normalisation

5. Augmentation de données et arrêt précoce

Augmentation et arrêt précoce



Augmentation

Transformations qui préservent l'étiquette : rotations, zooms, retournements, bruit, rétro-traduction...

Arrêt précoce

Arrêter quand la validation ne s'améliore plus depuis `patience` époques ; restaurer les meilleurs poids.

Tout combiner dans Keras

```
from tensorflow.keras import layers, regularizers
aug = tf.keras.Sequential([layers.RandomFlip("horizontal"), layers.RandomRotation(0.1)])
l2 = regularizers.l2(1e-4)
model = tf.keras.Sequential([layers.Input((32, 32, 3)), aug,
    layers.Conv2D(32, 3, padding='same', kernel_regularizer=l2, use_bias=False),
    layers.BatchNormalization(), layers.Activation('relu'), layers.MaxPooling2D(),
    layers.Conv2D(64, 3, padding='same', kernel_regularizer=l2, use_bias=False),
    layers.BatchNormalization(), layers.Activation('relu'), layers.MaxPooling2D(),
    layers.Flatten(), layers.Dense(128, activation='relu', kernel_regularizer=l2),
    layers.Dropout(0.5), layers.Dense(10, activation='softmax')])
early = tf.keras.callbacks.EarlyStopping(patience=5, restore_best_weights=True)
model.fit(x, y, epochs=100, validation_split=0.1, callbacks=[early])
```

À retenir

Weight decay (L2)	rétrécit les poids ; quasi systématique
L1	parcimonie
Dropout	désactivation aléatoire ; couches denses, RNN
Batch / Layer Norm	normalise les activations ; stabilise
Augmentation	enrichit les données
Arrêt précoce	toujours utile

Questions de révision

1. Entraînement 99 %, validation 78 % : trois actions ?
2. $p = 0{,}2$, $a = 5$: valeurs possibles de $\tilde{a}$ et espérance ?
3. Paramètres de `BatchNormalization` après 64 canaux ? (128 appris + 128 non appris)